MEESHO SUPERAPP CLONE • ENGINEERING AUDIT

Date: October 2026

Scope: Customer & Shopping Module

Stack: React 19 + Express + Prisma

Customer Panel: Pending Features & Gap Analysis Report

Comprehensive verification of implemented screens vs. missing backend endpoints, unpersisted workflows, and core e-commerce capabilities.

IMPLEMENTED CORE
~75%
Catalog, Cart, Orders, Reviews, Address

CRITICAL GAPS
4 Items
Return DB sync, Gateway, Hardcoded URLs

MISSING FEATURES
7 Items
Pincode check, Phone OTP, Seller Store

POLISH NEEDED
4 Screens
Flash sale DB query, Bottom nav roles

✅ 1. Completed & Functional Customer Modules

The following customer screens are already developed and integrated with the live PostgreSQL/Prisma database via REST APIs:

Screen & Route	Implemented Capabilities	File Location
Customer Home `/user-home`	Database products fetching, category carousel, hero banner slider, live search autocomplete, cart counter badge, instant wishlist toggle.	`HomeUserCustomer.jsx`
Categories Explorer `/explorer`, `/categories`	Split drawer category selection, subcategory chip filters, real-time products grid, direct Add-to-Cart.	`search_categories_explorer.jsx`
Search Results `/search`	Keyword search query handling, price & rating filters, sort options (Popularity, Price Low to High, High to Low).	`search_results.jsx`
Product Detail (PDP) `/product`	Image gallery, size selector with dynamic category size charts, color swatch selector, reviews preview, sticky CTA buttons.	`product_detail.jsx`
Shopping Cart `/cart`	Server-synchronized cart, quantity modifiers, item deletion, live coupon validation with DB coupons (`/api/v1/coupons`).	`shopping_cart.jsx`
Checkout Address `/address`	Saved delivery addresses from DB, Add New Address modal with form validation, default address selection.	`checkout_address_selection.jsx`
Checkout Payment `/payment`	Payment methods UI (COD, UPI, Card, NetBanking), Reseller margin toggle, backend order creation (`POST /orders`).	`checkout_payment.jsx`

Screen & Route	Implemented Capabilities	File Location
Customer Orders History `/orders`	Active / Delivered / Cancelled tabs, timeline tracking modal, order cancellation with live DB update, PDF Tax Invoice modal.	`CustomerOrders.jsx`
Ratings & Reviews `/reviews`, `/write-review`	Verified buyer validation (`checkCanReview`), star rating input, photo upload, fabric/fit sub-ratings.	`ratings_reviews.jsx` `write_a_review.jsx`
User Dashboard (Multi-Tab) `/user-dashboard1`	Unified consumer hub with Overview, Orders, Addresses, Wishlist, Profile settings, Wallet, and Affiliate links.	`user_web_dashboard1.jsx`

⚠️ **Critical Architecture Notice:** While the user interface covers the complete purchase funnel, there are several backend database disconnects (notably Return Requests and Payment Processing) where data remains trapped in local React state or `localStorage` instead of entering the server database.

🔴 2. Critical Backend & Data Integration Gaps (Must Fix First)

These are high-priority structural issues where frontend actions fail to persist in the PostgreSQL database:

Gap / Issue	Severity	Current Technical Flaw	Required Action
Return Request Database Disconnect	CRITICAL	`ReturnRequest.jsx` updates order status to `RETURN_REQUESTED` and saves data to browser `localStorage`, but does **NOT insert a row into the Prisma `ReturnRequest` table**.	Create `POST /api/v1/returns` endpoint in backend and link `handleSubmitReturn()` so Admin Panel's `/admin-returns` sees customer defect photos, reason & UPI.
Hardcoded Localhost URLs	CRITICAL	In `ReturnRequest.jsx` (lines 233 & 270), `http://localhost:5000/api/v1` is hardcoded instead of using Vite's `import.meta.env.VITE_BACKEND_URL`.	Replace with dynamic environment variable; otherwise network requests fail on production, tunnels, or LAN testing.
Payment Gateway Integration	CRITICAL	In `checkout_payment.jsx`, UPI, Debit Card, and NetBanking payments are **simulated with mock timeouts**. No real payment transaction happens.	Integrate Razorpay or Cashfree SDK with server-side order generation and webhook signature verification (COD is currently fully operational).
Review Helpful Votes Not Persisted	MEDIUM	On the reviews page, clicking "Helpful" upvotes is stored only in React component memory. Refreshing the browser resets the vote count.	Add a `POST /api/v1/reviews/:id/vote` route to store user upvotes in the database.

🟡 3. Missing Core Meesho Customer Features

Standard e-commerce features present on Meesho India that are currently absent from the customer panel:

Feature Name	Impact	Description & Missing Logic	Target Screen
Pincode Delivery Estimator	HIGH	Missing an *"Enter Pincode to check delivery date & COD availability"* widget on the PDP. Customers cannot check transit duration or eligibility prior to checkout.	`product_detail.jsx`
Phone Number + OTP Login	HIGH	Meesho's primary sign-in mechanism is SMS/WhatsApp OTP. Current system only provides Email/Password and Google OAuth.	`UserLogin.jsx`
Supplier Storefront View	MEDIUM	Clicking *"Sold by: [Supplier Name]"* on the product page does not open a dedicated seller profile page with all products from that merchant.	`product_detail.jsx` ➔ `/store/:id`
Similar / Recommended Products	MEDIUM	The bottom of the Product Detail page lacks a dynamic recommendation algorithm (matching categorySlug or shared tags from the database).	`product_detail.jsx`
Customer Notifications Feed	MEDIUM	The existing `/notifications` center is configured for resellers (margin credits, withdrawals). Missing customer alerts (Dispatched, Out for Delivery, Wishlist Price Drops).	`NotificationCenter.jsx`
Live Delivery GPS Map for Customer	LOW	Driver app has live route GPS maps, but customer's order tracking modal only displays linear text milestones without rider position display.	`CustomerOrders.jsx`
Order Support / Ticket Redirection	LOW	No one-click "Need Help with this Order" button on orders that passes the `orderId` directly to the support center ticket modal.	`CustomerOrders.jsx` ➔ `/support-center`

🟢 4. Frontend Polish & Mock/Static Pages

Screens that exist in the navigation catalog but currently render hardcoded mock data:

Screen / Route	Current State & Required Polish	File Path
Flash Sale Landing Page `/flash`	Uses static `initialProducts` array with hardcoded items. Needs to fetch live database products with `discountPercentage >= 50`.	`flash_sale_landing_page.jsx`
Curated Spotlight `/spotlight`	Contains static fashion lookbooks; header back button incorrectly points to `reseller` instead of customer home.	`curated_spotlight.jsx`
Flutter Luxe Theme `/luxe`	Concept theme page with mock static products. Can be linked to database or maintained as an alternate aesthetic showcase.	`home_screen_flutter_luxe.jsx`
Bottom Nav Role Consistency `/explorer`, `/orders`, `/wishlist`	Several customer pages render `<AppBottomNav />` without explicitly passing `role="customer"`, causing the Reseller "Earnings" tab to appear instead of "Wishlist".	`AppBottomNav.jsx`

🚀 5. Recommended Priority Implementation Roadmap

Structured 4-phase rollout to achieve 100% production readiness for the Customer Panel:

PHASE 1 • PRIORITY 1 — IMMEDIATE

Fix Return Request Backend & Admin Sync

Build `POST /api/v1/returns` endpoint in backend. Insert into Prisma `ReturnRequest` table so Admin Panel displays uploaded defect photos, reasons, and UPI details. Fix localhost URLs.

PHASE 2 • PRIORITY 2 — HIGH

Pincode Estimator & Seller Store

Implement Pincode lookup on Product Detail Page with standard delivery date calculation. Add customer-facing Seller Storefront page (`/store/:supplierName`).

PHASE 3 • PRIORITY 3 — MEDIUM

Dynamic Flash Sale & Bottom Nav Fix

Connect `/flash` to real discounted database products. Pass `role="customer"` across all customer pages so bottom dock shows correct tabs (Wishlist instead of Earnings).

PHASE 4 • PRIORITY 4 — FINISHING

Payment Gateway SDK & OTP Login

Integrate live Razorpay/Cashfree checkout modal with server signature validation. Add optional Firebase Phone OTP authentication widget for mobile-first users.